In [ ]:

LIST (list)

```
Sequence of items denoted with '[]' and separated with ','.

It is ordered(indexed with specific position), mutable(items can
be changed) and heterogenous(supports multiple data-type).
```

In [194…
```
l = [20,2.5,'cat',True]
print(l)
```

```
[20, 2.5, 'cat', True]
```

In [196…
```
type(l)
```

Out[196… list

In [198…
```
#indexing syntax : var[index]
l[3]
```

Out[198… True

In [200…
```
l[-1]
```

Out[200… True

In [ ]:

In [202…
```
#slicing
l[0:3:2]
```

Out[202… [20, 'cat']

In [204…
```
l[::]
```

Out[204… [20, 2.5, 'cat', True]

In [206…
```
l[::-1]
```

Out[206… [True, 'cat', 2.5, 20]

In [ ]:

List Methods

Adding Items to a List

- var.append(obj) : add item to the end of the list
- var.insert(ind,obj) : add item to the given index of the list
- var.extend([sequence]) : add items from the given sequence at the end of the list

Removing Items from the List

- var.remove(obj) : removes the first matched instance of object from the list
- var.pop(index) : it pop or delete the item of the given index from the list
- var.clear() : empty the list

Miscellaneous methods

- var.copy() : copy the item
- var.count(obj)
- var.index(obj)
- var.sort() : sort or arrange in ascending order
- var.sort(reverse=True) : arrange in descending order
- var.reverse() or var[: : -1] can be use to reverse the list

In [208…
```
l = [25,60,35,45,35,98]
print(l)
```
```
[25, 60, 35, 45, 35, 98]
```

In [210…
```
#adding items to a list
#append
print(l)
l.append(100)
print(l)
```
```
[25, 60, 35, 45, 35, 98]
[25, 60, 35, 45, 35, 98, 100]
```

In [212…
```
#insert
print(l)
l.insert(2,55)
print(l)
```
```
[25, 60, 35, 45, 35, 98, 100]
[25, 60, 55, 35, 45, 35, 98, 100]
```

In [214…
```
#extend
print(l)
l.extend([10,20,30])
print(l)
```
```
[25, 60, 55, 35, 45, 35, 98, 100]
[25, 60, 55, 35, 45, 35, 98, 100, 10, 20, 30]
```

In [ ]:
```
```

In [216…
```
#removing elements from a list
#pop
print(l)
l.pop()
print(l)
```
```
[25, 60, 55, 35, 45, 35, 98, 100, 10, 20, 30]
[25, 60, 55, 35, 45, 35, 98, 100, 10, 20]
```

In [218…
```
print(l)
l.pop(2)
print(l)
```

```
[25, 60, 55, 35, 45, 35, 98, 100, 10, 20]
[25, 60, 35, 45, 35, 98, 100, 10, 20]
```

In [220…
```
#remove
print(l)
l.remove(35)
print(l)
```

```
[25, 60, 35, 45, 35, 98, 100, 10, 20]
[25, 60, 45, 35, 98, 100, 10, 20]
```

In [222…
```
#clear
print(l)
l.clear()
print(l)
```

```
[25, 60, 45, 35, 98, 100, 10, 20]
[]
```

In [1]:
```
l = [35,90,78,56,20]
print(l)
```

```
[35, 90, 78, 56, 20]
```

In [3]:
```
#Miscellaneous methods in list
#copy
print(l)
m = l.copy()
print(m)
```

```
[35, 90, 78, 56, 20]
[35, 90, 78, 56, 20]
```

In [5]:
```
#count
l.count(56)
```

Out[5]: 1

In [7]:
```
#index
l.index(56)
```

Out[7]: 3

In [9]:
```
#sort - ascending order
print(l)
l.sort()
print(l)
```

```
[35, 90, 78, 56, 20]
[20, 35, 56, 78, 90]
```

In [11]:
```
print(l)
l.sort(reverse=True) #Descending order
print(l)
```

```
[20, 35, 56, 78, 90]
[90, 78, 56, 35, 20]
```

In [13]:
```
a = ['cat','gun','doll','sky']
a.sort()
print(a)
```

```
['cat', 'doll', 'gun', 'sky']
```

In [15]:
```
a = [10,89,'cat','gun','doll','sky']
a.sort()
print(a)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[15], line 2
      1 a = [10,89,'cat','gun','doll','sky']
----> 2 a.sort()
      3 print(a)

TypeError: '<' not supported between instances of 'str' and 'int'
```

In [17]:
```
a = ['10','cat','89','5','30','gun','doll','sky']
a.sort()
print(a)
```

```
['10', '30', '5', '89', 'cat', 'doll', 'gun', 'sky']
```

In [19]:
```
#reverse
a = [45,78,10,89]
print(a)
a.reverse()
print(a)
```

```
[45, 78, 10, 89]
[89, 10, 78, 45]
```

In [41]:

In [ ]:

Tuple (tuple)

An ordered but immutable(which cannot change size or permanent) collection of items.

It is denoted by parenthsis '()' and separated by ','.

In [21]:
```
t = (25,4.5,'dog',False)
print(t)
```

```
(25, 4.5, 'dog', False)
```

In [23]:
```
type(t)
```

Out[23]: tuple

In [25]:
```
#indexing
t[-2]
```

Out[25]: 'dog'

In [27]:
```
#slicing
t[::]
```

Out[27]: (25, 4.5, 'dog', False)

In [29]:
```
t[::-1]
```

Out[29]: (False, 'dog', 4.5, 25)

Tuple Methods

In [31]:
```
#count
t.count(25)
```

Out[31]: 1

In [33]:
```
#index
t.index('dog')
```

Out[33]: 2

In [35]:
```
t = (25,4.5,'dog',[1,2,3],False)
print(t)
```

```
(25, 4.5, 'dog', [1, 2, 3], False)
```

In [37]:
```
type(t)
```

Out[37]: tuple

In [39]:
```
t[3]
```

Out[39]: [1, 2, 3]

In [41]:
```
type(t[3])
```

Out[41]: list

In [43]:
```
print(t)
t[3].append(4)
print(t)
```

```
(25, 4.5, 'dog', [1, 2, 3], False)
(25, 4.5, 'dog', [1, 2, 3, 4], False)
```

In [ ]:
```

```

Set (set)

Unordered but mutable collection of unique items.

Denoted with curly brackets '{}' and separated by ','.

In [45]:
```
s = {35,True,2.4,'cat',40}
print(s)
```

```
{'cat', True, 2.4, 35, 40}
```

In [47]:
```
type(s)
```

Out[47]: set

In [49]:
```
a = {1,1,1,1,1,4,4,4,4,5,5,5,3,3,3}
print(a)
```

```
{1, 3, 4, 5}
```

In [ ]:

set methods

In [51]:
```
s
```

Out[51]: {2.4, 35, 40, True, 'cat'}

In [53]:
```
#add
print(s)
s.add(100)
print(s)
```

```
{'cat', True, 2.4, 35, 40}
{'cat', True, 2.4, 35, 100, 40}
```

In [55]:
```
a = {1,3,2,5,4,6,9}
b = {2,4,8}
```

In [57]:
```
#union
a.union(b)
```

Out[57]: {1, 2, 3, 4, 5, 6, 8, 9}

In [59]:
```
#intersection
a.intersection(b)
```

Out[59]: {2, 4}

In [61]:
```
#difference
a.difference(b)   #A-B
```

Out[61]: {1, 3, 5, 6, 9}

In [63]:
```
b.difference(a)   #B-A
```

Out[63]: {8}

In [65]:
```
#issuperset
a.issuperset(b)
```

Out[65]: False

In [69]:
```
#issubset
a.issubset(b)
```

Out[69]: False

In [71]:
```
#isdisjoint
a.isdisjoint(b)
```

Out[71]: False

In [73]:
```
#pop
print(s)
s.pop()
print(s)
```

```
{'cat', True, 2.4, 35, 100, 40}
{True, 2.4, 35, 100, 40}
```

In [75]:
```
print(s)
s.pop()
print(s)
```

```
{True, 2.4, 35, 100, 40}
{2.4, 35, 100, 40}
```

In [77]:
```
#remove
print(s)
s.remove(100)
print(s)
```

```
{2.4, 35, 100, 40}
{2.4, 35, 40}
```

In [79]:
```
#clear
print(s)
s.clear()
print(s)
```

```
{2.4, 35, 40}
set()
```

Dictionary (dict)

An ordered, mutable collection of key and value pairs.

Denoted with curly brackets '{}' and key must be unique.

key and value pairs are separated with comma and key and value are separated with colon

In [89]:
```
d = {"Name":"Dev", "Skill":"No Behen", "Course":"AIML"}
print(d)
```

```
{'Name': 'Dev', 'Skill': 'No Behen', 'Course': 'AIML'}
```

In [83]:
```
type(d)
```

Out[83]: dict

Dictionary methods

In [91]:
```
#update
print(d)
d.update({"Marks":99.9})
print(d)
```

```
{'Name': 'Dev', 'Skill': 'No Behen', 'Course': 'AIML'}
{'Name': 'Dev', 'Skill': 'No Behen', 'Course': 'AIML', 'Marks': 99.9}
```

In [93]:
```
print(d)
d.update({"Skill":"Bhaichara On Top"})
print(d)
```

```
{'Name': 'Dev', 'Skill': 'No Behen', 'Course': 'AIML', 'Marks': 99.9}
{'Name': 'Dev', 'Skill': 'Bhaichara On Top', 'Course': 'AIML', 'Marks': 99.9}
```

In [95]:
```
# keys()
d.keys()
```

Out[95]: dict_keys(['Name', 'Skill', 'Course', 'Marks'])

In [97]:
```
# values()
d.values()
```

Out[97]: dict_values(['Dev', 'Bhaichara On Top', 'AIML', 99.9])

In [99]:
```
# items()
d.items()
```

Out[99]: dict_items([('Name', 'Dev'), ('Skill', 'Bhaichara On Top'), ('Course', 'AIML'), ('Marks', 99.9)])

In [101…
```
d['Name']
```

Out[101… 'Dev'

In [103…
```
# get()
d.get('Name')
```

Out[103… 'Dev'

In [105…
```
d.get('Skill')
```

Out[105… 'Bhaichara On Top'

In [107…
```
#popitem
print(d)
d.popitem()
print(d)
```

```
{'Name': 'Dev', 'Skill': 'Bhaichara On Top', 'Course': 'AIML', 'Marks': 99.9}
{'Name': 'Dev', 'Skill': 'Bhaichara On Top', 'Course': 'AIML'}
```

In [109…
```
#pop
print(d)
```

```
d.pop('Skill')
print(d)
```

```
{'Name': 'Dev', 'Skill': 'Bhaichara On Top', 'Course': 'AIML'}
{'Name': 'Dev', 'Course': 'AIML'}
```

In [111…

```
#clear
print(d)
d.clear()
print(d)
```

```
{'Name': 'Dev', 'Course': 'AIML'}
{}
```

In [ ]:

Operators : Operators are the symbols which are used to perform operations on values and variables

Types of Operators in Python :

- Arithmatic Operators
- Assignment Operators
- Comparision/Relational Operators
- Logical Operators
- Identity Operators
- Membership Operators

Arithmatic Operators

```
Arithmetic operators are used to perform mathematical operations
like addition, subtraction, multiplication, etc.
```

Operator	Name	Example
+	Addition	x + y
-	Subtraction	x - y
*	Multiplication	x * y
/	Division	x / y
%	Modulus	x % y
**	Exponentiation	x ** y
//	Floor division	x // y

In [113…
```
a = 11
b = 2
```

In [115…
```
#addition
a + b
```

Out[115… 13

In [117…
```
#subtraction
a - b
```

Out[117… 9

In [119…
```
#multiplication
a * b
```

Out[119… 22

In [121…
```
#division
a / b
```

Out[121… 5.5

In [123…
```
#floor division
a // b
```

Out[123… 5

In [125…
```
#modulo
a % b
```

Out[125… 1

In [129…
```
#power
a ** b
```

Out[129… 121

In [133…
```
2 ** 5 ** 3
```

Out[133… 42535295865117307932921825928971026432

In [135…
```
2 ** 125
```

Out[135… 42535295865117307932921825928971026432

In [ ]:

Assignment Operators

Assignment operators are used to assign values to variables.

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
//=	x //= 3	x = x // 3
**=	x **= 3	x = x ** 3

In [147…
```
a = 10
a
```

Out[147… 10

In [149…
```
a += 1
a
```

Out[149… 11

In [151…
```
a -= 2
a
```

Out[151… 9

In [153…
```
a *= 3
a
```

Out[153… 27

In [155…
```
a /= 2
a
```

Out[155… 13.5

In [157…
```
a //= 2
a
```

Out[157… 6.0

In [159…
```
a %= 5
a
```

Out[159… 1.0

In [161…
```
a **= 3
a
```

Out[161… 1.0

In [ ]:

Comparision/Relational Operators

Comparison operators compare two values/variables and return a boolean result: True or False

Operator	Name	Example
==	Equal	x == y
!=	Not equal	x != y
>	Greater than	x > y
<	Less than	x < y
>=	Greater than or equal to	x >= y
<=	Less than or equal to	x <= y

In [163… 6 == 6

Out[163… True

In [165… 5 != 5

Out[165… False

In [167… 10 > 5

Out[167… True

In [169… 5 < 10

Out[169… True

In [171… 20 >= 30

Out[171… False

In [173… 30 <= 30

Out[173… True

In [ ]:

Logical Operators

Logical operators are used to check whether an expression is True or False. They are used in decision-making.

Operator	Description	Example
and	Returns True if both statements are true	x < 5 and x < 10
or	Returns True if one of the statements is true	x < 5 or x < 4
not	Reverse the result, returns False if the result is true	not(x < 5 and x < 10)

In [175…
```
#AND
10>5 and 5<10
```

Out[175… True

In [177…
```
10>5 and 5>10
```

Out[177… False

In [179…
```
10<5 and 5<10
```

Out[179… False

In [181…
```
10>5 and 5>10
```

Out[181… False

In [183…
```
#OR
10>5 or 5<10
```

Out[183… True

In [185…
```
10>5 or 5>10
```

Out[185… True

In [187…
```
10<5 or 5<10
```

Out[187… True

In [189…
```
10<5 or 5>10
```

Out[189… False

In [191…
```
#NOT
not 10>5
```

Out[191… False

In [193…
```
not 5>10
```

Out[193… True

Special Operators

Python Identity Operators

Identity operators are used to compare the objects, not if they are equal, but if they are actually the same object, with the same memory location:

Operator	Description	Example
is	Returns True if both variables are the same object	x is y
is not	Returns True if both variables are not the same object	x is not y

In [1]:
```
#identity operator - 'is' and 'is not'
a = 10
type(a)
```

Out[1]:
```
int
```

In [3]:
```
id(a)  #to check the memory location
```

Out[3]:
```
140710176439000
```

In [25]:
```
a = 256
b = 256
```

In [27]:
```
a is b
```

Out[27]:
```
True
```

In [29]:
```
id(a)
```

Out[29]:
```
140710176446872
```

In [31]:
```
id(b)
```

Out[31]:
```
140710176446872
```

In [39]:
```
m = 'abcd123_'
n = 'abcd123_'

m is n
```

Out[39]:
```
True
```

In [41]:
```
x = [1,2,3,4]
y = [1,2,3,4]
```

In [43]:
```
x is y
```

Out[43]:
```
False
```

In [45]:
```
x is not y
```

Out[45]:
```
True
```

Python Membership Operators

Membership operators are used to test if a sequence is presented in an object:

Operator	Description	Example
in	Returns True if a sequence with the specified value is present in the object	x in y
not in	Returns True if a sequence with the specified value is not present in the object	x not in y

In [47]:
```
#membership operator - 'in' and 'not in'
'A' in 'apple'
```

Out[47]: False

In [53]:
```
'A' not in 'apple'
```

Out[53]: True

In [55]:
```
10 in [25,67,10,78]
```

Out[55]: True

In [ ]:
```

```

String Operators

In [60]:
```
# '+' use to concatenate/joining strings
'cat🐱' + 'dog🐶'
```

Out[60]: 'cat🐱dog🐶'

In [64]:
```
# '*' use to replicate/duplicate the string
'🍜' * 5
```

Out[64]: '🍜🍜🍜🍜🍜'

Type Casting

Conversion of data types

In [66]:
```
a = '123'
print(a)
type(a)
```

```
123
```

Out[66]: str

In [68]:
```
i = int(a)
print(i)
```

```
type(i)
```

```
123
```

Out[68]: int

In [70]:
```
f = float(i)
print(f)
type(f)
```

```
123.0
```

Out[70]: float

In [72]:
```
c = complex(f)
print(c)
type(c)
```

```
(123+0j)
```

Out[72]: complex

In [74]:
```
a = "Hello Everyone"
print(a)
type(a)
```

```
Hello Everyone
```

Out[74]: str

In [78]:
```
l = list(a)
print(l)
type(l)
```

```
['H', 'e', 'l', 'l', 'o', ' ', 'E', 'v', 'e', 'r', 'y', 'o', 'n', 'e']
```

Out[78]: list

In [80]:
```
t = tuple(a)
print(t)
type(t)
```

```
('H', 'e', 'l', 'l', 'o', ' ', 'E', 'v', 'e', 'r', 'y', 'o', 'n', 'e')
```

Out[80]: tuple

In [82]:
```
s = set(a)
print(s)
type(s)
```

```
{'y', 'e', 'r', 'n', 'H', 'E', ' ', 'v', 'o', 'l'}
```

Out[82]: set

In [84]:
```
dict(a)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[84], line 1
----> 1 dict(a)

ValueError: dictionary update sequence element #0 has length 1; 2 is required
```

In [86]:
```
b = [("Name","Army"), ('Plan','WW3')]
```

```
d = dict(b)
print(d)
type(d)
```

```
{'Name': 'Army', 'Plan': 'WW3'}
```

Out[86]:
```
dict
```

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]: